

TUGAS SO 5 SISTEM BOOKING HOTEL



Nama : Ageng Setyo Nugroho

Nim : 2410651037

Kelas : B

Tanggal : Kamis, 13 November 2025

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

SOAL PEMROGRAMAN

Tugas Kuliah

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

Hotel memiliki 5 kamar yang bisa di-booking

Ada 10 customer yang mencoba booking secara bersamaan (10 threads)

Setiap customer akan:

1. Mencoba booking kamar (butuh waktu random 1-3 detik)
2. Jika berhasil, "menginap" selama 5 detik
3. Check-out dan melepaskan kamar
4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi

Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

Requirements:

1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)

3. Tampilkan informasi:

1. Customer ID berapa yang berhasil booking
2. Kamar mana yang ditempati
3. Berapa kamar yang masih tersedia
4. Customer yang gagal booking

4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas)

Contoh Output yang Diharapkan:

SISTEM BOOKING HOTEL

Total Kamar: 5

[10:30:15] 🧑 Customer-1 mencoba booking...

[10:30:15] 🧑 Customer-2 mencoba booking...

[10:30:15] ✅ Customer-1 berhasil booking! (4 kamar tersisa)

[10:30:16] ✅ Customer-2 berhasil booking! (3 kamar tersisa)

[10:30:16] 🧑 Customer-3 mencoba booking...

[10:30:16] ❌ Customer-10 gagal booking (kamar penuh), mencoba lagi...

[10:30:20] 📅 Customer-1 check-out (4 kamar tersisa)

...

CODE

```
// File: hotel_booking.c
```

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define TOTAL_ROOMS 5
```

```
#define TOTAL_CUSTOMERS 10
```

```
sem_t available_rooms;    // Semaphore untuk jumlah kamar tersedia
```

```
pthread_mutex_t print_lock; // Mutex untuk melindungi operasi print
```

```
int rooms_available = TOTAL_ROOMS; // Variabel global (dilindungi semaphore & mutex)
```

```
// Fungsi untuk mendapatkan waktu sekarang dalam format [HH:MM:SS]
```

```
void get_time(char *buffer) {
```

```
    time_t raw;
```

```
    struct tm *timeinfo;
```

```
    time(&raw);
```

```
    timeinfo = localtime(&raw);
```

```
    sprintf(buffer, "[%02d:%02d:%02d]", timeinfo->tm_hour, timeinfo->tm_min, timeinfo->tm_sec);
```

```
}
```

```
// Fungsi yang dijalankan oleh setiap thread (customer)
```

```
void* customer_booking(void* arg) {
```

```
    int id = *(int*)arg;
```

```
    int attempts = 0;
```

```
    char now[20];
```

```

while (attempts < 3) {

    attempts++;

    get_time(now);

    pthread_mutex_lock(&print_lock);

    printf("%s 🚫 Customer-%d mencoba booking (Percobaan ke-%d)...\n", now, id,
attempts);

    pthread_mutex_unlock(&print_lock);

    sleep(rand() % 3 + 1); // waktu random 1–3 detik

    // Coba dapatkan kamar

    if (sem_trywait(&available_rooms) == 0) {

        pthread_mutex_lock(&print_lock);

        rooms_available--;

        get_time(now);

        printf("%s ✅ Customer-%d berhasil booking! (%d kamar tersisa)\n", now, id,
rooms_available);

        pthread_mutex_unlock(&print_lock);

        sleep(5); // waktu menginap

        // Check-out

```

```

    sem_post(&available_rooms);

    pthread_mutex_lock(&print_lock);

    rooms_available++;

    get_time(now);

    printf("%s 📅 Customer-%d check-out (%d kamar tersisa)\n", now, id,
rooms_available);

    pthread_mutex_unlock(&print_lock);


    pthread_exit(NULL); // selesai
} else {

    pthread_mutex_lock(&print_lock);

    get_time(now);

    printf("%s ❌ Customer-%d gagal booking (kamar penuh), mencoba lagi...\n",
now, id);

    pthread_mutex_unlock(&print_lock);

    sleep(2); // tunggu sebelum mencoba lagi

}

}

pthread_mutex_lock(&print_lock);

get_time(now);

printf("%s 😞 Customer-%d menyerah setelah 3 kali gagal.\n", now, id);

pthread_mutex_unlock(&print_lock);

```

```

pthread_exit(NULL);
}

int main() {

pthread_t customers[TOTAL_CUSTOMERS];

int ids[TOTAL_CUSTOMERS];

srand(time(NULL));

// Inisialisasi semaphore & mutex

sem_init(&available_rooms, 0, TOTAL_ROOMS);

pthread_mutex_init(&print_lock, NULL);

printf("=== 🏨 SISTEM BOOKING HOTEL ===\n");

printf("Total Kamar: %d\n\n", TOTAL_ROOMS);

// Membuat thread untuk setiap customer

for (int i = 0; i < TOTAL_CUSTOMERS; i++) {

ids[i] = i + 1;

pthread_create(&customers[i], NULL, customer_booking, &ids[i]);

usleep(100000); // jeda 0.1 detik antar-customer

```



```

    }

    // Tunggu semua customer selesai

    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        pthread_join(customers[i], NULL);
    }

    printf("\n=== Semua Customer Selesai ===\n");

    // Hancurkan semaphore dan mutex

    sem_destroy(&available_rooms);

    pthread_mutex_destroy(&print_lock);

    return 0;
}

```

Cara Kompilasi dan Menjalankan

```

gcc hotel_booking.c -o hotel_booking -pthread

./hotel_booking

```

1.1 Analisis Masalah

- **Pemahaman Soal:**

Dalam simulasi ini, terdapat 10 customer yang ingin memesan kamar di hotel yang hanya memiliki 5 kamar.

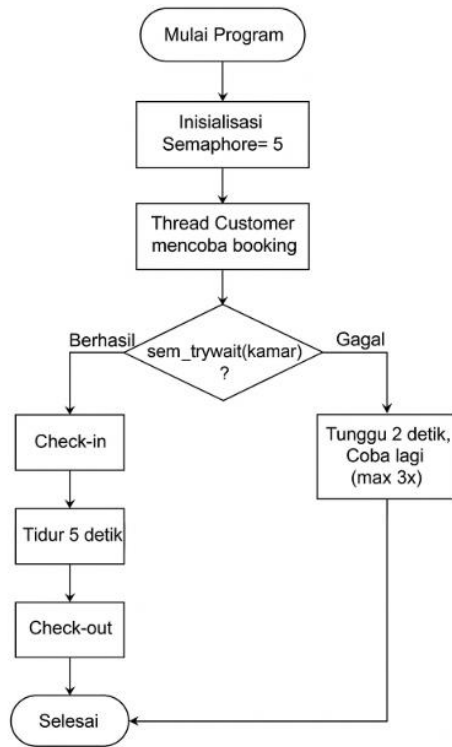
Karena semua customer berjalan secara bersamaan (multithreading), perlu digunakan sinkronisasi agar tidak terjadi perebutan kamar atau overbooking.
- **Identifikasi Masalah Sinkronisasi:**
 1. Dua atau lebih customer bisa mengakses data kamar secara bersamaan.
 2. Output bisa berantakan jika semua thread mencetak ke layar secara bersamaan.
- **Desain Solusi:**
 1. Gunakan Semaphore untuk mengatur jumlah kamar (maksimal 5).
 2. Gunakan Mutex untuk melindungi bagian output (printf) agar rapi.
 3. Setiap customer memiliki batas 3 kali percobaan booking.

1.2 Implementasi

Penjelasan Algoritma:

1. Inisialisasi semaphore dengan nilai 5 (jumlah kamar).
2. Setiap thread (customer) mencoba `sem_trywait()`.
3. Jika berhasil, customer menginap selama 5 detik lalu `sem_post()` saat keluar.
4. Jika gagal (kamar penuh), customer menunggu 2 detik dan mencoba lagi maksimal 3 kali.
5. Semua operasi `printf()` dilindungi dengan `pthread_mutex_lock()` agar output tidak tumpang tindih.

Flowchart:



Potongan Kode Penting:

```
if (sem_trywait(&available_rooms) == 0) {
    pthread_mutex_lock(&print_lock);
    printf("✔ Customer-%d berhasil booking! (%d kamar tersisa)\n", id,
rooms_available-1);
    pthread_mutex_unlock(&print_lock);

    sleep(5); // waktu menginap
    sem_post(&available_rooms); // check-out
} else {
    pthread_mutex_lock(&print_lock);
    printf("✘ Customer-%d gagal booking (kamar penuh), mencoba lagi...\n",
id);
    pthread_mutex_unlock(&print_lock);
    sleep(2);
}
```

Alasan Pemilihan Mekanisme:

- Semaphore cocok untuk mengatur jumlah resource terbatas (kamar).
- Mutex digunakan hanya untuk melindungi akses ke resource tunggal (layar/console output).

1.3 Testing & Output

- Test Case 1: 5 kamar, 10 customer

✓ Hasil: Maksimal 5 customer berhasil booking bersamaan, sisanya menunggu.

Setelah 5 detik, kamar yang kosong langsung diisi oleh customer lain.

- Test Case 2: Ubah kamar menjadi 1

✓ Hasil: Customer bergantian masuk satu per satu.

- Test Case 3: Ubah jumlah customer menjadi 3

✓ Hasil: Semua berhasil booking tanpa antrian.

Contoh Output :

[illegible]

1.4 Analisis Hasil

- Apakah sesuai ekspektasi?
Ya, hanya 5 customer yang dapat booking bersamaan, sisanya menunggu atau menyerah. Tidak ada overbooking.
- Masalah yang Ditemui:
Kadang urutan output berubah karena sifat paralelisme, tapi hasil tetap konsisten karena ada mutex.
- Performa:
Program efisien dan stabil. Semua sinkronisasi berjalan dengan benar tanpa deadlock.
- Pelajaran yang Diperoleh:
Penggunaan semaphore dan mutex penting agar sistem multi-threaded tetap aman dan konsisten.

1.5 Kesimpulan

1. Sinkronisasi dibutuhkan agar thread tidak mengakses data bersama secara bersamaan.
2. Mutex digunakan untuk melindungi akses tunggal, semaphore untuk sumber daya terbatas.
3. Tanpa sinkronisasi, terjadi race condition dan output tidak valid.
4. Praktikum ini melatih pemahaman sinkronisasi pada kasus nyata seperti sistem booking hotel.